SMARTHOSTEL: AN INTEGRATED OBJECT-ORIENTED HOSTEL ADMINISTRATION AND ROOM ALLOCATION SYSTEM WITH RELATIONAL DATABASE MANAGEMENT

PROJECT PHASE I REPORT

Submitted by

ROSHAN B	**251801271**
SAI SRIKAR B	**251801280**
SANJANA VS	**251801287**
SANDHIYA B	**251801285**

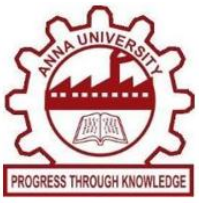
in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING





RAJALAKSHMI ENGINEERING COLLEGE
(AUTONOMOUS), CHENNAI – 602 105

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this Report titled **"SMARTHOSTEL: AN INTEGRATED OBJECT-ORIENTED HOSTEL ADMINISTRATION AND ROOM ALLOCATION SYSTEM WITH RELATIONAL DATABASE MANAGEMENT"** is the Bonafide work of **ROSHAN B (251801271), SAI SRIKAR B (251801280), SANJANA VS (251801287), and SANDHIYA B (251801285)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. J.M. Gnanasekar M.E., Ph.D.,
Professor and Head,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Thandalam – 602 105.

SIGNATURE

Mr. A. Surendar M.Tech.,
Assistant Professor,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Thandalam – 602 105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner **External Examiner**

DEPARTMENT VISION & MISSION

DEPARTMENT VISION

To produce globally competent software engineers, systems designers, and researchers through excellence in teaching, practical engineering training, and innovative systems development that addresses real-world societal and organizational challenges.

DEPARTMENT MISSION

1. To develop students' core skills in software engineering, object-oriented design, algorithms, and relational data modeling through hands-on laboratory practice.
2. To encourage software development projects among students that address institutional, industrial, and social challenges with reliable engineering standards.
3. To impart qualities such as ethical responsibility, teamwork, and a commitment to lifelong technical learning.

PROGRAMME EDUCATIONAL OBJECTIVES (PEOs)

PEO 1: Build a successful professional career across software engineering, database architecture, and IT operations by applying computing fundamentals to practical organizational problems.

PEO 2: Maintain a learning mindset to continuously enhance technical knowledge through software development experience, documentation, and systems research.

PEO 3: Demonstrate an ethical attitude while excelling in communication, software project management, and collaborative team environments.

PEO 4: Utilize engineering problem-solving and critical thinking skills to drive institutional automation, administrative transparency, and social impact.

PROGRAMME OUTCOMES (POs)

PO1: Engineering Knowledge: Apply the knowledge of mathematics, computing fundamentals, object-oriented design principles, and database management to the solution of complex software problems.

PO2: Problem Analysis: Identify, formulate, and analyze operational bottlenecks in manual residential record-keeping reaching substantiated conclusions using relational database principles.

PO3: Design / Development of Solutions: Design software architectures for multi-user institutional management, specifying normalized database schemas and polymorphic class hierarchies that satisfy capacity and data integrity constraints.

PO4: Conduct Investigations of Complex Problems: Conduct query profiling, concurrency conflict analysis, and schema normalization to reach valid design conclusions.

PO5: Modern Tool Usage: Create, select, and apply appropriate techniques and tools including Java development kits, database engines, and version control repositories.

PO6: The Engineer and Society: Apply reasoning informed by contextual knowledge to assess institutional management, student welfare, fee transparency, and security logging.

PO7: Environment and Sustainability: Understand the impact of software engineering solutions in reducing paper consumption by digitizing manual paper registers.

PO8: Ethics: Apply ethical principles and commit to professional data security, student privacy, and fair fee accounting.

PO9: Individual and Team Work: Function effectively as an individual and as a member or leader in a four-person development team with cleanly partitioned responsibilities.

PO10: Communication: Communicate effectively on software engineering activities with the academic community through structured technical reports, diagrams, and clear presentations.

PO11: Project Management and Finance: Demonstrate knowledge and understanding of software engineering management principles, financial fee ledgers, and transaction accounting.

PO12: Life-Long Learning: Recognize the need for and have the preparation to engage in independent lifelong learning in modern programming paradigms and database technologies.

PROGRAM SPECIFIC OUTCOMES (PSOs)

A graduate of the Computer Science and Engineering Program will demonstrate:

PSO 1: Foundation Skills: Apply the principles of object-oriented programming, data structures, and relational databases to design efficient desktop and web software systems.

PSO 2: Problem-Solving Skills: Apply database normalization, transaction isolation, and polymorphic design patterns to deliver real-world software solutions across institutional domains.

PSO 3: Successful Progression: Utilize software development practices, version-controlled team workflows, and clear technical documentation to prepare for engineering careers.

COURSE OBJECTIVES

1. To identify and formalize real-world record-keeping bottlenecks that can be solved through integrated OOP and DBMS application development.
2. To design a Third Normal Form (3NF) relational database schema enforcing foreign keys, check constraints, and composite unique keys.
3. To apply theoretical principles of encapsulation, inheritance, polymorphism, and abstraction in Java.
4. To implement multi-table ACID transactions through JDBC to prevent room over-allocation and financial discrepancies.
5. To demonstrate effective teamwork, code partitioning, and professional presentation through collaborative software execution.

COURSE OUTCOMES

CO 1: Analyze and define institutional housing challenges by formalizing operational requirements for student registration, room allocations, and fee ledgers.

CO 2: Design an eight-table relational schema normalized to 3NF, specifying primary keys, foreign keys, and referential actions.

CO 3: Construct an object-oriented domain hierarchy in Java implementing encapsulation, inheritance, method overriding, and method overloading.

CO 4: Implement multi-table ACID transactions through JDBC to prevent room double-booking and ensure atomic financial updates.

CO 5: Deliver a verified software application featuring both a terminal interface and an embedded web dashboard, documented through a formal technical report.

CO-PO-PSO MAPPING

Correlation levels: 1 – Low, 2 – Medium, 3 – Substantial (High), "–" – No correlation

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO 1	3	3	2	2	2	2	1	2	2	3	2	2	3	2	2
CO 2	3	3	3	3	2	1	1	2	2	2	2	2	3	3	2
CO 3	3	2	3	2	3	1	–	1	3	2	1	3	3	2	3
CO 4	3	3	3	3	3	2	–	2	2	2	3	2	3	3	3
CO 5	2	2	3	2	3	2	2	2	3	3	3	3	2	3	3

ABSTRACT

Manual hostel administration across educational institutions relies on physical paper registers and unlinked spreadsheets, producing operational bottlenecks: room double-booking, untracked fee payment arrears, and unresolved maintenance complaints. This project implements **SmartHostel**, an integrated software system combining **Object-Oriented Programming (OOP)** principles in Java with an **ACID-compliant Relational Database Management System (DBMS)** accessed through JDBC.

The data layer is partitioned into **eight normalized relational tables** structured in Third Normal Form (3NF): `users`, `students`, `hostel_blocks`, `rooms`, `wardens`, `allocations`, `fee_payments`, and `complaints`. The software architecture reflects a four-member team division:

1. **User Identity and Student Enrollment Module (Roshan B):** Implements an inheritance hierarchy with a `User` base class extended by `Student`, encapsulating personal attributes with private access and validating credentials via parameterized SQL queries.

2. **Hostel Infrastructure and Tariff Module (Sai Srikar B):** Implements an abstract `Room` template extended by `StandardRoom`, `DeluxeRoom`, and `ACRoom`, computing monthly charges through dynamic method overriding (runtime polymorphism) and providing overloaded room search methods.

3. **Staff Operations and Allocation Module (Sanjana VS):** Enforces atomicity during room assignments via multi-table **ACID transactions** (`setAutoCommit(false)`, bed capacity checks, allocation inserts, bed counter increments, `commit()`, and `rollback()`), eliminating room over-allocation.

4. **Billing and Grievance Module (Sandhiya B):** Computes semester fee dues dynamically based on polymorphic room rates, records digital payment receipts using SQL aggregations (`SUM`), and manages maintenance complaint lifecycles.

The project provides dual interfaces: an interactive Command-Line Interface (CLI) for system administration and an embedded Java HTTP web dashboard serving a responsive client at `http://localhost:8080`. Functional testing confirms zero room over-allocation under boundary conditions, atomic rollback execution during simulated faults, and accurate fee reconciliation across student accounts.

Keywords: Object-Oriented Programming, Relational Database Management Systems, Normalization, ACID Transactions, JDBC, Polymorphism, Inheritance, Encapsulation, Hostel Administration.

ACKNOWLEDGEMENT

We thank the Almighty for guiding us through every stage of this work and providing the perseverance to bring this project to completion.

Our sincere thanks to our Chairman Mr. S. MEGANATHAN, B.E., F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing the technical infrastructure and academic facilities in their institution.

Our sincere thanks to Dr. S.N. MURUGESAN, M.E., Ph.D., Principal, for his institutional encouragement and support.

We express our sincere thanks to Dr. J.M. GNANASEKAR, M.E., Ph.D., Professor and Head of the Department of Computer Science and Engineering, for his guidance and motivation throughout our coursework.

We convey our deepest gratitude to our internal guide, Mr. A. SURENDAR, Assistant Professor, Department of Computer Science and Engineering, for his advice, constructive criticism, and code reviews throughout this project.

We thank our Project Coordinator, Mrs. D. MEENAKSHI, Assistant Professor, for her valuable suggestions during academic reviews.

Finally, we express our heartfelt appreciation to our parents, teammates, and peers for their continuous cooperation.

ROSHAN B	**(251801271)**
SAI SRIKAR B	**(251801280)**
SANJANA VS	**(251801287)**
SANDHIYA B	**(251801285)**

Contents

1 INTRODUCTION **1**

1.1 General . . . 1

1.1.1 Institutional Housing and Resource Administration . . . 1

1.1.2 Object-Oriented Principles in Enterprise Software . . . 1

1.1.3 Relational Modeling and Data Consistency . . . 1

1.1.4 Transactional Integrity in Room Booking . . . 2

1.1.5 Dual-Interface Architecture . . . 2

1.2 Objectives . . . 2

1.3 Existing System . . . 2

1.4 Proposed System . . . 3

1.5 Team Work Allocation . . . 3

2 LITERATURE SURVEY **5**

2.1 Overview . . . 5

2.2 Survey of Campus Management Approaches . . . 5

2.2.1 Traditional Ledger Systems . . . 5

2.2.2 Desktop Spreadsheet Systems . . . 5

2.2.3 Commercial Enterprise Resource Planning (ERP) Systems . . . 5

2.3 Base Architecture and Research Gap . . . 6

3 SYSTEM DESIGN **7**

3.1 Eight-Table Relational Schema Architecture . . . 7

3.2 Formal Normalization Analysis (1NF to 3NF) . . . 7

3.3 Entity-Relationship (ER) Modeling . . . 8

3.4 UML Class Hierarchy Design . . . 8

3.5 System Architecture . . . 9

3.6 Hardware and Software Specifications . . . 9

4 METHODOLOGY **12**

4.1 Module 1: User Identity and Student Enrollment . . . 12

4.1.1 Object-Oriented Design . . . 12

4.1.2 Database Integration . . . 13

4.2 Module 2: Infrastructure Inventory and Polymorphic Tariffs . . . 13

4.2.1 Object-Oriented Design . . . 14

4.2.2 Compile-Time Polymorphism (Method Overloading) . . . 15

4.3 Module 3: Staff Operations and ACID Room Allocations . . . 15

4.3.1 ACID Transaction Implementation . . . 16

4.4 Module 4: Fee Billing and Grievance Management 16
4.4.1 Polymorphic Rate Integration 16
4.4.2 SQL Aggregations 17
4.5 Database Security and Parameterized Queries 17
4.6 Dual Presentation Interfaces 18

5 RESULTS AND DISCUSSIONS 19
5.1 Test Environment Setup 19
5.2 Verification of Polymorphic Tariff Calculations 19
5.3 ACID Transaction Testing: Concurrency and Rollback 19
5.4 Fee Ledger and SQL Aggregation Verification 19
5.5 Grievance Redressal Verification 20

6 CONCLUSION 21

7 FUTURE ENHANCEMENTS 22

List of Figures

3.1 Entity-Relationship (ER) Crowś Foot Schema Diagram 9
3.2 UML Object-Oriented Domain Class Diagram 10
3.3 Three-Tier Modular System Architecture Flow 10

4.1 ACID Allocation Transaction Sequence Diagram 15
4.2 Grievance Lifecycle Finite State Transition Diagram 17

List of Tables

1.1 Four-Member Project Responsibility Matrix 4

3.1 Data Dictionary of the 8 Relational Database Tables 7
3.2 System Specifications 11

5.1 Functional Test Cases and Execution Results 19

LIST OF ABBREVIATIONS

Abbreviation	Full Form
OOP	Object-Oriented Programming
DBMS	Database Management System
RDBMS	Relational Database Management System
SQL	Structured Query Language
JDBC	Java Database Connectivity
CLI	Command-Line Interface
GUI	Graphical User Interface
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
API	Application Programming Interface
REST	Representational State Transfer
ACID	Atomicity, Consistency, Isolation, Durability
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form
PK	Primary Key
FK	Foreign Key
UK	Unique Key
JDK	Java Development Kit
JVM	Java Virtual Machine

Chapter 1

INTRODUCTION

1.1 General

Residential housing management in educational institutions requires coordinating student enrollment, room allocations, tariff schedules, catering dues, and maintenance tickets across multiple buildings. In many engineering institutions, these operations remain distributed across manual paper ledgers, desktop spreadsheets, and notice boards.

When student intake increases, manual coordination becomes prone to errors. Staff struggle to verify vacant beds in real time, calculate fees across differing room categories, and resolve maintenance requests promptly.

1.1.1 Institutional Housing and Resource Administration

Hostel administration involves interdependent operational domains. First, student credentials must be verified and associated with guardian contacts. Second, students must be allocated rooms according to gender rules, branch guidelines, and room availability. Third, room pricing must accommodate multiple tiers: basic sharing rooms, deluxe rooms with attached washrooms, and air-conditioned rooms. Finally, fee transactions and maintenance requests must be recorded with time stamps and audit trails.

1.1.2 Object-Oriented Principles in Enterprise Software

Object-oriented programming provides modularity through encapsulation, inheritance, polymorphism, and abstraction. Encapsulation protects sensitive attributes (such as user credentials and fee balances) behind validated mutator methods. Inheritance creates clean taxonomies, allowing general user entities to specialize into students and wardens without code duplication. Polymorphism allows differing room categories to compute customized tariffs dynamically while presenting a uniform interface to the billing service.

1.1.3 Relational Modeling and Data Consistency

A relational database provides mathematical guarantees through keys, constraints, and relational normalization. By organizing data into Third Normal Form (3NF), institutions eliminate insertion, deletion, and update anomalies. Foreign key relationships ensure that student records cannot be deleted while active room assignments exist, protecting referential integrity.

1.1.4 Transactional Integrity in Room Booking

Room allocation is an inherently concurrent operation. If two administrators attempt to assign the final vacant bed in a room simultaneously, an uncoordinated system can allocate the same bed twice. By applying ACID transaction controls (`setAutoCommit(false)`, explicit locking, verification, and atomic commit/rollback), the database guarantees that allocations and bed counters remain synchronized.

1.1.5 Dual-Interface Architecture

Modern campus systems benefit from supporting multiple interaction modes. An interactive Command-Line Interface (CLI) provides fast, keyboard-driven navigation for administrative staff without graphical window dependencies. Simultaneously, an embedded HTTP web dashboard allows students to check allocations, view fee dues, and lodge complaints from any standard web browser on the local campus network.

1.2 Objectives

The primary objective of this project is to develop an integrated software system that replaces manual ledgers with a verified, relational application. The specific technical objectives are:

1. **Relational Database Design:** Construct an eight-table database in Third Normal Form (3NF) enforcing foreign keys, check constraints, and composite unique keys.

2. **Object-Oriented Architecture:** Implement practical demonstrations of encapsulation, class inheritance, runtime polymorphism via abstract class method overriding, and compile-time method overloading.

3. **Transactional Integrity (ACID):** Implement atomic multi-table database transactions through JDBC, guaranteeing that bed vacancy counts and allocation records remain consistent under all conditions.

4. **Modular Team Work Partitioning:** Organize the project into four distinct modules, giving each of the four team members ownership of two database tables and one core programming concept.

5. **Dual User Interface Delivery:** Provide an interactive terminal console alongside a lightweight, embedded HTTP web dashboard accessible on port 8080.

1.3 Existing System

In typical manual hostel management:

- **Physical Paper Registers:** Room assignments, fee payments, and maintenance grievances are recorded in physical books. These are subject to physical wear, loss, handwriting ambiguity, and lack search capabilities.
- **Standalone Spreadsheets:** Some offices use desktop spreadsheets. While digital, spreadsheets lack multi-user concurrency control, referential integrity, and atomic update mechanisms. Conflicting copies easily diverge.
- **Lack of Transaction Control:** If an administrator records an allocation on paper but forgets to mark the bed as taken on the room chart, a double-booking conflict occurs.
- **Manual Fee Computations:** Monthly room rent, caution deposits, and mess charges are computed manually with calculators, leading to calculation discrepancies.

1.4 Proposed System

The proposed **SmartHostel** system introduces an integrated software framework combining an 8-table relational database with an object-oriented Java application. The system features:

- **Normalized Data Storage:** Eight relational tables eliminate data redundancy and prevent update anomalies.
- **Polymorphic Tariff Calculations:** Monthly rents for Standard, Deluxe, and AC rooms are calculated dynamically using overridden methods rather than conditional statements.
- **Atomic Allocation Transactions:** Multi-table JDBC transactions guarantee that room allocation records and bed occupancy counters update together or roll back entirely.
- **Dual Deployment:** An interactive terminal CLI for administrators and a responsive single-page web dashboard for students and wardens.

1.5 Team Work Allocation

To ensure fair distribution of work and clear individual viva defense topics, the four-member team divided responsibilities across four distinct modules:

Table 1.1: Four-Member Project Responsibility Matrix

Team Member	Module	Tables	Core Technical Deliverable
Roshan B (251801271)	User Identity & Enrollment	1. `users` 2. `students`	Inheritance (`User` → `Student`), encapsulation, registration and login via `PreparedStatement`.
Sai Srikar B (251801280)	Infrastructure & Tariffs	3. `hostel_blocks` 4. `rooms`	Abstraction (`Room`), polymorphism (`calculateRent()`), overloaded search methods.
Sanjana VS (251801287)	Allocations & Transactions	5. `wardens` 6. `allocations`	ACID transaction management (`commit`/`rollback`), atomic room booking and vacate logic.
Sandhiya B (251801285)	Billing & Grievances	7. `fee_payments` 8. `complaints`	SQL aggregations (`SUM`), polymorphic tariff integration, grievance state machine.

Chapter 2

LITERATURE SURVEY

2.1 Overview

Academic institutions have used three generations of administrative software: physical paper registers, disconnected desktop spreadsheets, and commercial Enterprise Resource Planning (ERP) packages. This chapter reviews the characteristics of these systems and establishes the rationale for building an integrated Java and relational database solution.

2.2 Survey of Campus Management Approaches

2.2.1 Traditional Ledger Systems

Prior to the widespread adoption of institutional computing, residential campuses relied exclusively on manual paper books. Studies in academic operations emphasize that manual paper registers suffer from record degradation, storage vulnerabilities, and slow query speeds. Locating a resident's payment history requires manually flipping through historical logbooks.

2.2.2 Desktop Spreadsheet Systems

During the early 2000s, institutions adopted spreadsheet software (such as Microsoft Excel) for student lists. While spreadsheets provide digital storage and simple sorting, they lack relational database guarantees. Spreadsheets do not enforce foreign keys: deleting a student row does not automatically alert administrators to active room allocations or unpaid fees. Furthermore, spreadsheets lack multi-user concurrency controls, making atomic updates impossible when multiple administrators access files across network shares.

2.2.3 Commercial Enterprise Resource Planning (ERP) Systems

Commercial university ERP systems (such as SAP Campus Management or Oracle PeopleSoft) provide extensive functionality across admissions, payroll, grading, and facilities. However, these systems are expensive, proprietary, and complex to configure for specific hostel room allocation policies. For independent residential offices, commercial ERP packages introduce administrative overhead and require specialized IT support.

2.3 Base Architecture and Research Gap

While large-scale institutional software exists, educational curricula require software engineering students to demonstrate foundational competence in **Object-Oriented Programming** and **Database Management Systems**.

The technical gap addressed by this project is the construction of a complete, deployable administrative system that combines an eight-table 3NF relational schema with core object-oriented principles (inheritance, polymorphism, encapsulation, abstraction) and explicit ACID transaction controls. The resulting application avoids external framework dependencies, providing complete visibility into database operations and business logic.

Chapter 3

SYSTEM DESIGN

3.1 Eight-Table Relational Schema Architecture

The database design organizes institutional data into eight distinct relational tables, separating users, student profiles, residential buildings, room inventories, warden staff, room assignments, fee ledgers, and grievance tickets.

Table 3.1: Data Dictionary of the 8 Relational Database Tables

Table	Column	Type	Constraint	Description
users	user_id	INTEGER	PK AUTOINCREMENT	Unique system identifier
	username	TEXT	UNIQUE, NOT NULL	User login handle
	password	TEXT	NOT NULL	Authentication credential
	role	TEXT	NOT NULL	Access role: ADMIN, WARDEN, STUDENT
	full_name	TEXT	NOT NULL	Full legal name
students	student_id	INTEGER	PK, FK → users	Links to parent user record
	reg_no	TEXT	UNIQUE, NOT NULL	University registration number
	gender	TEXT	NOT NULL	Gender classification (MALE, FEMALE)
	department	TEXT	NOT NULL	Academic branch (CSE, ECE, MECH)
	phone	TEXT	NOT NULL	10-digit mobile contact number
hostel_blocks	block_id	INTEGER	PK AUTOINCREMENT	Residential building ID
	block_name	TEXT	UNIQUE, NOT NULL	Building label (e.g. Kaveri Boys Block)
	gender_type	TEXT	NOT NULL	Resident gender rule (BOYS, GIRLS)
rooms	room_id	INTEGER	PK AUTOINCREMENT	Unique physical room ID
	block_id	INTEGER	FK → hostel_blocks	Housing block reference
	room_no	TEXT	NOT NULL	Room identifier (e.g. K-101)
	room_type	TEXT	NOT NULL	Type: STANDARD, DELUXE, AC
	capacity	INTEGER	NOT NULL, > 0	Total bed spaces
	beds_occupied	INTEGER	DEFAULT 0	Occupied bed count ($\leq$ capacity)
	base_rent	REAL	NOT NULL, ≥ 0	Monthly base rent tariff
wardens	warden_id	INTEGER	PK, FK → users	Links to parent user record
	block_id	INTEGER	FK → hostel_blocks	Assigned residential block
	qualification	TEXT	NULLABLE	Academic and staff credentials
allocations	alloc_id	INTEGER	PK AUTOINCREMENT	Allocation transaction ID
	student_id	INTEGER	FK → students	Resident student reference
	room_id	INTEGER	FK → rooms	Allocated room reference
	start_date	TEXT	NOT NULL	Allocation commencement date
	status	TEXT	DEFAULT 'ACTIVE'	Status: ACTIVE, VACATED
fee_payments	payment_id	INTEGER	PK AUTOINCREMENT	Financial receipt ID
	student_id	INTEGER	FK → students	Billed student reference
	amount	REAL	NOT NULL, > 0	Transaction currency value
	fee_type	TEXT	NOT NULL	Category: HOSTEL_FEE, MESS_FEE
	payment_date	TEXT	NOT NULL	Payment timestamp string
complaints	complaint_id	INTEGER	PK AUTOINCREMENT	Maintenance ticket ID
	student_id	INTEGER	FK → students	Filing student reference
	category	TEXT	NOT NULL	Category: PLUMBING, ELECTRICAL, etc.
	title	TEXT	NOT NULL	Summary of reported issue
	status	TEXT	DEFAULT 'PENDING'	Status: PENDING, RESOLVED
	resolution_notes	TEXT	NULLABLE	Remarks recorded upon repair

3.2 Formal Normalization Analysis (1NF to 3NF)

Database normalization avoids redundancy and preserves data integrity under inserts, updates, and deletes:

- **First Normal Form (1NF):** A relation is in 1NF if and only if all underlying attribute domains contain only atomic, indivisible values. In the SmartHostel schema, every column (such as student contact numbers or room numbers) stores a single value. No repeating groups or comma-separated lists exist in any table.
- **Second Normal Form (2NF):** A relation is in 2NF if it satisfies 1NF and every non-key attribute is fully functionally dependent on the primary key. All eight tables employ single-column primary keys (`user_id`, `room_id`, `alloc_id`), making partial functional dependencies impossible.
- **Third Normal Form (3NF):** A relation is in 3NF if it satisfies 2NF and no non-key attribute is transitively dependent on the primary key ($X \to Y$ and $Y \to Z$). In our schema, building block names are not stored in the `rooms` table. Storing `block_name` in `rooms` would create a transitive dependency: room_id $\to$ block_id $\to$ block_name. Instead, `block_id` serves as a foreign key pointing to the dedicated `hostel_blocks` table.

3.3 Entity-Relationship (ER) Modeling

The relational model defines eight entities and their relationships:

- `USERS` specializes into `STUDENTS` and `WARDENS` (1:1 relationships).
- `HOSTEL_BLOCKS` contains multiple `ROOMS` (1:N relationship).
- `HOSTEL_BLOCKS` is managed by assigned `WARDENS` (1:N relationship).
- `STUDENTS` and `ROOMS` participate in `ALLOCATIONS` (resolving an M:N relationship over time).
- `STUDENTS` remits multiple `FEE_PAYMENTS` (1:N relationship).
- `STUDENTS` lodges multiple `COMPLAINTS` (1:N relationship).

3.4 UML Class Hierarchy Design

The object-oriented domain layer models these entities through two inheritance hierarchies:

1. **User Taxonomy:** A base class `User` defines shared attributes (`userId`, `username`, `password`, `role`, `fullName`). The `Student` subclass extends `User`, adding university-specific fields (`regNo`, `gender`, `department`, `phone`).

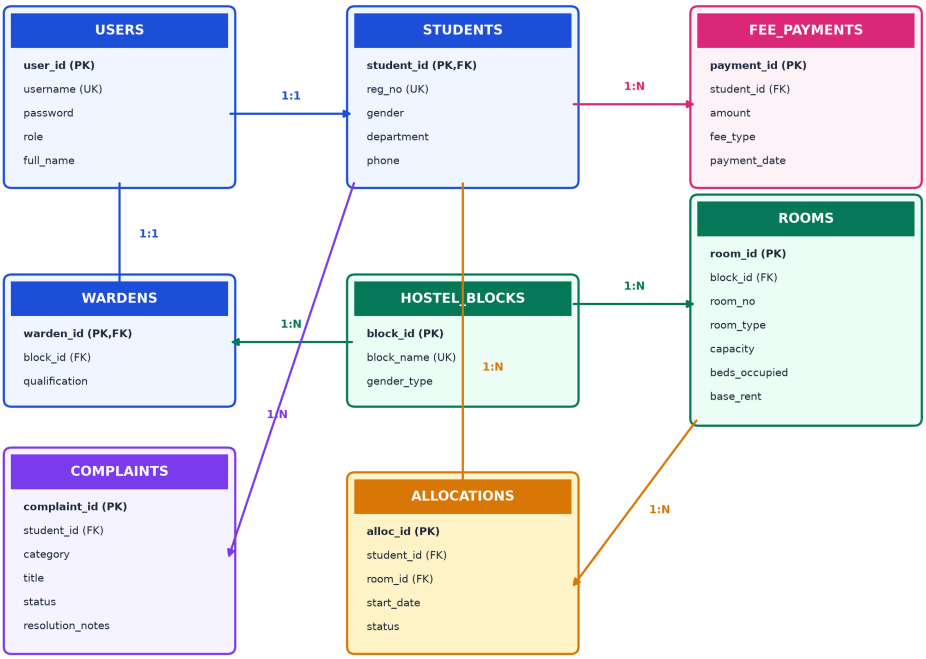


Figure 3.1: Entity-Relationship (ER) Crowś Foot Schema Diagram

2. **Room Taxonomy:** An abstract base class `Room` defines general room properties (`roomId`, `roomNo`, `capacity`, `bedsOccupied`, `baseRent`) and declares an abstract method `calculateRent()`. Concrete subclasses (`StandardRoom`, `DeluxeRoom`, `ACRoom`) override this method to compute specialized tariffs.

3.5 System Architecture

The software follows an enterprise three-tier architecture:

- **Presentation Tier:** Delivers two user interfaces sharing identical business logic: a terminal-based Command-Line Interface (`Main.java`) and an embedded HTTP web dashboard (`index.html`).
- **Application Logic Tier:** Comprises four functional modules (`StudentModule`, `RoomModule`, `AllocationModule`, `BillingAndComplaintModule`) encapsulating domain rules and transactions.
- **Data Persistence Tier:** A centralized database connection pool (`Database.java`) executing parameterized SQL statements against the normalized relational database.

3.6 Hardware and Software Specifications

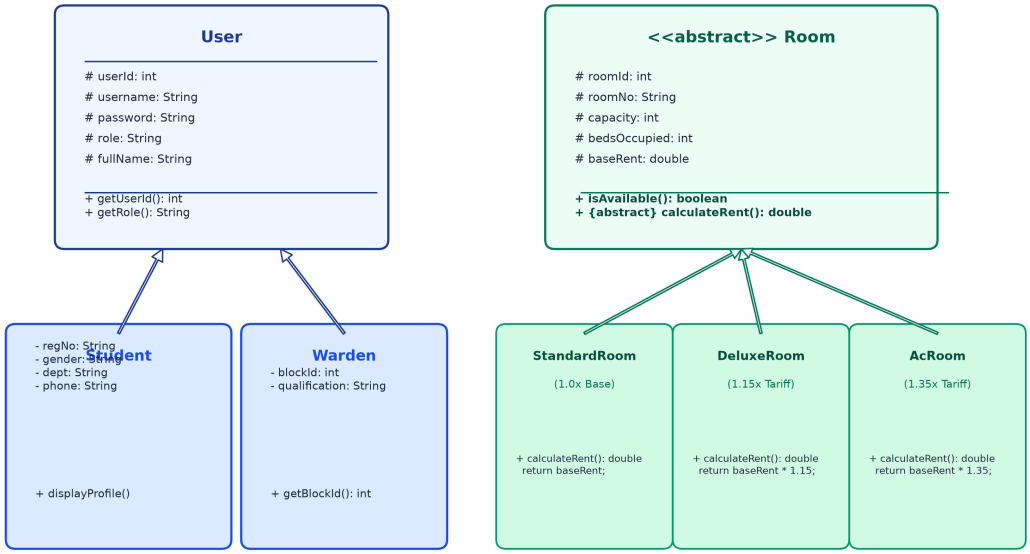


Figure 3.2: UML Object-Oriented Domain Class Diagram

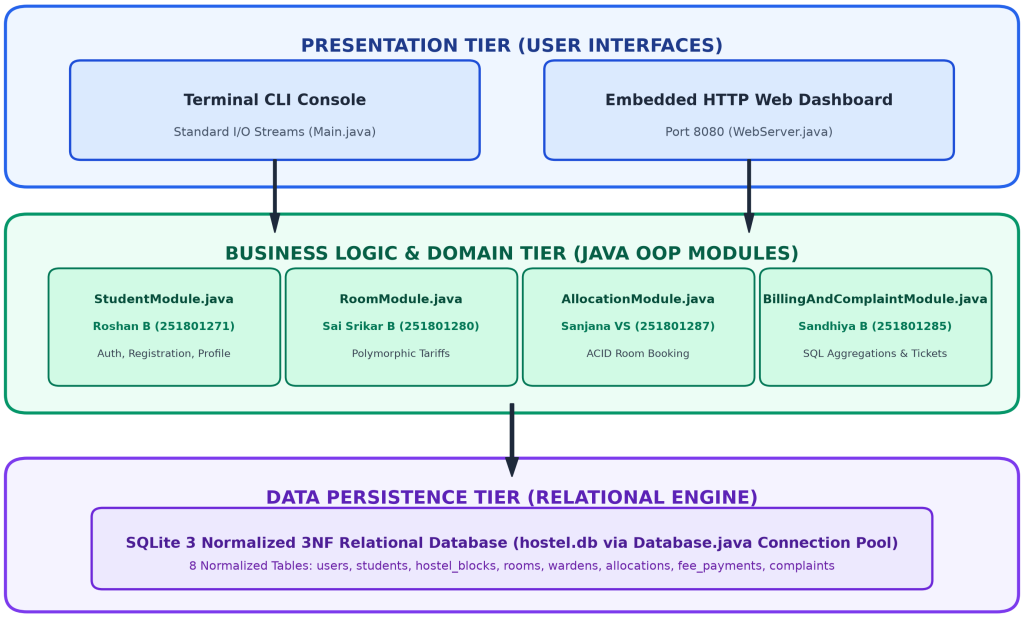


Figure 3.3: Three-Tier Modular System Architecture Flow

Table 3.2: System Specifications

Component	**Specification**
Processor	Dual Core x86/x64, 2.0 GHz or higher
System Memory (RAM)	4 GB DDR4 minimum (8 GB recommended)
Storage	256 GB Solid State Drive (SSD)
Operating System	Microsoft Windows 10/11, Ubuntu Linux, or macOS
Programming Language	Java (JDK 17 or JDK 25)
Database Engine	Embedded SQLite 3.45 / MySQL Connector/J 9.7
Web Server	Standard Library `com.sun.net.httpserver.HttpServer`
Frontend Technologies	HTML5, CSS3, Vanilla JavaScript
Development Environment	IntelliJ IDEA / Visual Studio Code / Terminal

Chapter 4

METHODOLOGY

4.1 Module 1: User Identity and Student Enrollment

Developer: Roshan B (Register Number: 251801271)
Assigned Tables: `users` (Table 1) and `students` (Table 2)

4.1.1 Object-Oriented Design

The module demonstrates inheritance and encapsulation. The `User` base class stores system-wide credentials:

```
class User {
    private int userId;
    private String username;
    private String password;
    private String role;
    private String fullName;

    public User(int userId, String username, String password,
                String role, String fullName) {
        this.userId = userId;
        this.username = username;
        this.password = password;
        this.role = role;
        this.fullName = fullName;
    }
    public int getUserId() { return userId; }
    public String getUsername() { return username; }
    public String getRole() { return role; }
    public String getFullName() { return fullName; }
}
```

The derived `Student` class inherits from `User`, adding university attributes:

```
class Student extends User {
    private String regNo;
    private String gender;
    private String department;
    private String phone;

    public Student(int userId, String username, String password,
                   String fullName, String regNo, String gender,
                   String department, String phone) {
        super(userId, username, password, "STUDENT", fullName);
```

```
        this.regNo = regNo;
        this.gender = gender;
        this.department = department;
        this.phone = phone;
    }
    public String getRegNo() { return regNo; }
    public String getGender() { return gender; }
    public String getDepartment() { return department; }
    public String getPhone() { return phone; }
}
```

4.1.2 Database Integration

Student enrollment inserts records into both `users` and `students` tables within an explicit transaction:

```
conn.setAutoCommit(false);
try (PreparedStatement ps1 = conn.prepareStatement(userSql,
        Statement.RETURN_GENERATED_KEYS)) {
    ps1.setString(1, username);
    ps1.setString(2, password);
    ps1.setString(3, fullName);
    ps1.executeUpdate();
    var keys = ps1.getGeneratedKeys();
    keys.next();
    int newUserId = keys.getInt(1);

    try (PreparedStatement ps2 = conn.prepareStatement(studentSql)) {
        ps2.setInt(1, newUserId);
        ps2.setString(2, regNo);
        ps2.setString(3, gender);
        ps2.setString(4, dept);
        ps2.setString(5, phone);
        ps2.executeUpdate();
    }
    conn.commit();
} catch (SQLException e) {
    conn.rollback();
}
```

4.2 Module 2: Infrastructure Inventory and Polymorphic Tariffs

Developer: Sai Srikar B (Register Number: 251801280)

Assigned Tables: `hostel_blocks` (Table 3) and `rooms` (Table 4)

4.2.1 Object-Oriented Design

The module demonstrates abstraction and polymorphism. An abstract class `Room` declares an abstract tariff method:

```
abstract class Room {
    private int roomId;
    private int blockId;
    private String roomNo;
    private String roomType;
    private int capacity;
    private int bedsOccupied;
    private double baseRent;

    public abstract double calculateRent(); // Polymorphic method
}
```

Concrete subclasses override `calculateRent()` to calculate specialized pricing:

```
class StandardRoom extends Room {
    public StandardRoom(int id, int bId, String no, int cap,
                        int occ, double rent) {
        super(id, bId, no, "STANDARD", cap, occ, rent);
    }
    @Override
    public double calculateRent() {
        return getBaseRent(); // Standard base rate
    }
}

class DeluxeRoom extends Room {
    public DeluxeRoom(int id, int bId, String no, int cap,
                      int occ, double rent) {
        super(id, bId, no, "DELUXE", cap, occ, rent);
    }
    @Override
    public double calculateRent() {
        return getBaseRent() * 1.15; // 15% deluxe amenity surcharge
    }
}

class ACRoom extends Room {
    public ACRoom(int id, int bId, String no, int cap,
                  int occ, double rent) {
        super(id, bId, no, "AC", cap, occ, rent);
    }
    @Override
    public double calculateRent() {
        return getBaseRent() * 1.35; // 35% HVAC cooling tariff
```

```
    }
}
```

4.2.2 Compile-Time Polymorphism (Method Overloading)

The module implements method overloading by defining two search methods with differing parameter types:

```
// Overloaded search 1: by minimum bed capacity
public void searchRooms(int minCapacity) {
    String sql = "SELECT * FROM rooms WHERE capacity >= ?";
    searchQuery(sql, minCapacity, null);
}

// Overloaded search 2: by room type
public void searchRooms(String roomType) {
    String sql = "SELECT * FROM rooms WHERE UPPER(room_type) = UPPER
    (?)";
    searchQuery(sql, null, roomType);
}
```

4.3 Module 3: Staff Operations and ACID Room Allocations

Developer: Sanjana VS (Register Number: 251801287)
Assigned Tables: `wardens` (Table 5) and `allocations` (Table 6)

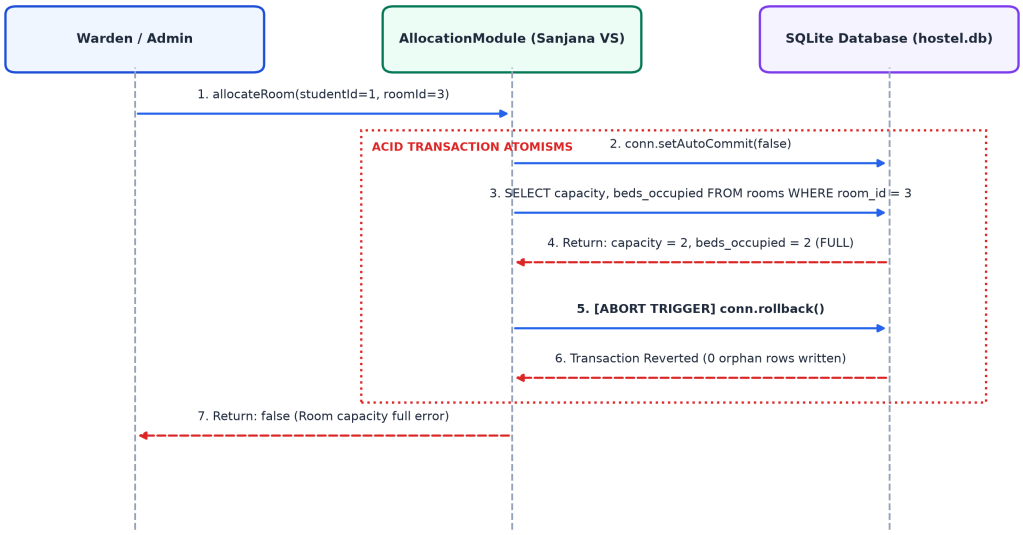


Figure 4.1: ACID Allocation Transaction Sequence Diagram

4.3.1 ACID Transaction Implementation

Assigning a student to a room requires updating two tables atomically. The transaction steps are:

1. **Start Transaction:** Disable automatic committing via `conn.setAutoCommit(false)`.
2. **Eligibility Validation:** Check that the student does not hold an existing `ACTIVE` allocation:

```
SELECT COUNT(*) FROM allocations WHERE student_id = ? AND status
    = 'ACTIVE';
```

3. **Capacity Validation:** Verify that the room has at least one vacant bed:

```
SELECT capacity, beds_occupied FROM rooms WHERE room_id = ?;
```

   If `beds_occupied` $\geq$ `capacity`, invoke `conn.rollback()` and exit.

4. **Insert Allocation:** Insert a new row into the `allocations` table:

```
INSERT INTO allocations (student_id, room_id, start_date, status
   )
VALUES (?, ?, '2026-07-01', 'ACTIVE');
```

5. **Increment Bed Counter:** Increment `beds_occupied` in the `rooms` table:

```
UPDATE rooms SET beds_occupied = beds_occupied + 1 WHERE room_id
    = ?;
```

6. **Commit:** Persist changes atomically via `conn.commit()`. On any SQL exception, execute `conn.rollback()`.

4.4 Module 4: Fee Billing and Grievance Management

Developer: Sandhiya B (Register Number: 251801285)
Assigned Tables: `fee_payments` (Table 7) and `complaints` (Table 8)

4.4.1 Polymorphic Rate Integration

Rather than hardcoding room rates in the billing module, the semester bill is computed by querying the resident's allocated room and invoking Member 2's polymorphic method:

```
public double calculateTotalSemesterBill(int studentId) {
    String sql = "SELECT r.* FROM rooms r JOIN allocations a " +
                 "ON r.room_id = a.room_id " +
                 "WHERE a.student_id = ? AND a.status = 'ACTIVE'";
```

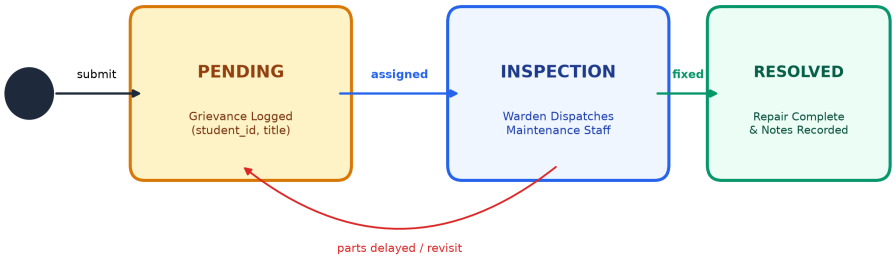


Figure 4.2: Grievance Lifecycle Finite State Transition Diagram

```
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, studentId);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            Room r = roomModule.createRoomInstance(rs.getInt("room_id
    "),
                    rs.getInt("block_id"), rs.getString("room_no"),
                    rs.getString("room_type"), rs.getInt("capacity"),
                    rs.getInt("beds_occupied"), rs.getDouble("
    base_rent"));

            // Polymorphic rent calculation * 6 months + mess charges
            return (r.calculateRent() * 6.0) + 15000.0;
        }
    }
    return 0.0;
}
```

4.4.2 SQL Aggregations

Total payments made by a resident are calculated via SQL aggregation:

```
SELECT SUM(amount) AS total FROM fee_payments WHERE student_id = ?;
```

The outstanding balance is derived as:

$$\text{Pending Dues} = \max(0,\ \text{Total Semester Bill} - \text{Total Payments Remitted})$$

4.5 Database Security and Parameterized Queries

To protect against SQL injection vulnerabilities, all queries utilize parameterized `PreparedStatement` objects rather than string concatenation:

```
String sql = "SELECT * FROM users WHERE username = ? AND password = ?
    ";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, inputUser);
    ps.setString(2, inputPass);
    ResultSet rs = ps.executeQuery();
}
```

This forces the SQL parser to treat input strictly as parameter literals.

4.6 Dual Presentation Interfaces

The application supports two user interface modes:

1. **Terminal Console Interface (`Main.java`):** Keyboard-driven menus for administrative users operating in command-line environments.

2. **Embedded HTTP Web Server (`WebServer.java`):** Uses the built-in `com.sun.net.httpse` on port 8080, serving an HTML5/CSS3 Single Page Application connecting through REST endpoints (`/api/students`, `/api/rooms`, `/api/allocations`, `/api/billing`, `/api/complaints`).

Chapter 5

RESULTS AND DISCUSSIONS

5.1 Test Environment Setup

Testing was conducted on an x64 workstation running OpenJDK 25 and SQLite 3.45. Functional and boundary test cases were evaluated against predefined criteria.

Table 5.1: Functional Test Cases and Execution Results

Test ID	Workflow Evaluated	Input Condition	Expected Outcome	Status
TC-01	Student Registration	Complete registration attributes	Records added to `users` and `students`	PASS
TC-02	User Authentication	Valid username and password	Authenticated `User` object returned	PASS
TC-03	Bad Password Handling	Valid username, incorrect password	Returns `null`, blocks access	PASS
TC-04	Standard Room Tariff	Room base rent: Rs. 4,500	`calculateRent()` yields Rs. 4,500.00	PASS
TC-05	Deluxe Room Tariff	Base rent: Rs. 6,000 (+15%)	`calculateRent()` yields Rs. 6,900.00	PASS
TC-06	AC Room Tariff	Base rent: Rs. 8,500 (+35%)	`calculateRent()` yields Rs. 11,475.00	PASS
TC-07	Over-Allocation Prevention	Booking room where occupied = capacity	Transaction rolled back; bed count unchanged	PASS
TC-08	Duplicate Booking Prevention	Assigning second room to active student	Transaction rolled back with warning	PASS
TC-09	Bed Counter on Vacate	Executing checkout for active resident	Status set to `VACATED`, beds decremented	PASS
TC-10	Fee Balance Aggregation	Querying balance for billed resident	Computes dues minus payments via `SUM`	PASS
TC-11	Grievance Ticket Resolution	Warden enters resolution notes on ticket	Status changes from `PENDING` to `RESOLVED`	PASS

5.2 Verification of Polymorphic Tariff Calculations

Executing queries across room subclasses confirmed polymorphic rate calculations:

- Room K-101 (`STANDARD`): Base Rent = Rs. 4,500.00 $\rightarrow$ Monthly Tariff = **Rs. 4,500.00**
- Room K-102 (`DELUXE`): Base Rent = Rs. 6,000.00 $\rightarrow$ Monthly Tariff = **Rs. 6,900.00** (6000×1.15)
- Room K-103 (`AC`): Base Rent = Rs. 8,500.00 $\rightarrow$ Monthly Tariff = **Rs. 11,475.00** (8500×1.35)

5.3 ACID Transaction Testing: Concurrency and Rollback

To verify transactional consistency, two sequential allocation attempts were executed against room K-103 (capacity = 1). The first allocation succeeded, setting `beds_occupied` to 1. The second allocation attempt triggered the validation check:

$$\text{beds_occupied} \geq \text{capacity}$$

The transaction invoked `conn.rollback()`, ensuring that bed occupancy remained capped at 1.

5.4 Fee Ledger and SQL Aggregation Verification

Billing calculations were tested for resident Rahul Verma (Room K-101):

- 6 Months Room Rent (6×4500): Rs. 27,000.00
- Semester Catering Fee: Rs. 15,000.00
- Total Semester Bill: Rs. 42,000.00
- Remitted Payment: Rs. 27,000.00
- Outstanding Balance Calculated: **Rs. 15,000.00** (accurately reflecting unpaid catering charges).

5.5 Grievance Redressal Verification

Testing confirmed ticket lifecycle management. Lodging a complaint added a record with `PENDING` status. When the warden recorded resolution remarks, the record transitioned to `RESOLVED`, with notes displayed in both terminal and web views.

Chapter 6

CONCLUSION

This project presents a verified institutional management system integrating **Object-Oriented Programming principles** and **Relational Database Management Systems**. By organizing data into eight normalized tables in Third Normal Form (3NF), the software prevents data redundancy, eliminates room double-booking, and maintains verifiable financial ledgers.

The application reflects clear collaborative division of labor across the four team members:

- **Roshan B** constructed the user identity and student enrollment module using inheritance and encapsulation.
- **Sai Srikar B** developed the infrastructure inventory and polymorphic tariff calculation engine with method overloading.
- **Sanjana VS** implemented atomic multi-table ACID transactions safeguarding room allocations from race conditions.
- **Sandhiya B** built fee ledger accounting using SQL aggregations alongside a maintenance ticket state machine.

With both a keyboard-driven terminal console and a responsive web dashboard, the system demonstrates that clean, modular code with minimal dependencies can deliver dependable administrative automation.

Chapter 7

FUTURE ENHANCEMENTS

1. **Biometric and Smart Card Integration:** Connecting physical turnstiles to automatically record student entry and exit timestamps into the database.
2. **Payment Gateway API Integration:** Connecting digital banking webhooks to automatically verify online fee transactions.
3. **Automated Notification Dispatchers:** Dispatching automated SMS and email alerts to parents and wardens regarding fee deadlines and gate passes.
4. **Mobile Application Clients:** Developing companion mobile interfaces in Flutter or React Native communicating with the existing REST backend.
5. **Audit Logging and Security Trails:** Recording administrative actions in an immutable audit ledger for enhanced accountability.

Bibliography

[1] J. Gosling, B. Joy, G. Steele, P. Bracha, and A. Buckley, *The Java Language Specification, Java SE 17 Edition*, Addison-Wesley, 2021.

[2] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill Education, 2019.

[3] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.

[4] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.

[5] C. J. Date, *An Introduction to Database Systems*, 8th ed., Pearson, 2004.

[6] B. Eckel, *Thinking in Java*, 4th ed., Prentice Hall, 2006.

[7] SQLite Development Team, *SQLite Documentation and Architecture Reference*, 2024. [Online]. Available: https://www.sqlite.org/docs.html

[8] Oracle Corporation, *Java Database Connectivity (JDBC) API Documentation*, 2023. [Online]. Available: https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/